AgentCore as the Sandbox in Agentic RL

AgentCore Runtime as the execution sandbox, integrated with slime

Kaige Yang
Senior Applied Scientist, AWS

September 10, 2026 · results cited through 2026-09-28

Agenda

01 Why sandboxes

What frontier labs run in sandboxes, and how they score it

02 Architecture & pipeline

Reference architecture, the five components, slime and its async rollout

03 Foundations of AgentCore

Runtime / session / microVM, CPU vs GPU, session lifecycle

04 Integrating into slime

The call path, one hook, the episode loop, the key code

05 The demos

4 main + side note + experimental: config, checker, measured results, status

06 Cross-cutting mechanisms

Episode shape, TITO, the `min()` law, one episode → many samples, resilience

Appendix (from slide 71): references.

© 2026, Amazon Web Services, Inc. or its affiliates.

In agentic RL a programmatic checker, not an LLM judge, turns a multi-turn episode into the reward

In plain English: the policy acts over many turns against an executable environment, and a **programmatic checker** — not an LLM judge — turns the outcome into a scalar reward (RLVR).

Scope: AgentCore = the execution **sandbox**; **slime** = the trainer; the **harness runs inside each episode's sandbox session**, the trainer keeps only a dispatch tier + adapters.

Roadmap (6 sections)

1. **Why sandboxes** — what frontier labs run in sandboxes, and how they score it.
2. **Architecture & pipeline** — reference architecture, the five components, slime + its async rollout.
3. **Foundations of AgentCore** — Runtime / session / microVM, CPU vs GPU compute, session lifecycle & launch/terminate.
4. **Integrating into slime** — the call path, one hook, the episode loop, and the key code.
5. **The demos** — all 6 at a glance (4 main + side note + experimental), then one slide per demo (config, checker, measured eval/latency, honest status, real use case).
6. **Cross-cutting mechanisms** — episode shape, TITO, the scaling `min()` law, one-episode→many-samples, and the resilience levers.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore Runtime is the sandbox: one isolated session per episode, wired into slime through one hook

- **Problem.** Agentic RL runs **untrusted model output** in an isolated environment thousands of times per run and scores it with a **programmatic checker** — the *sandbox* layer, not the GPU, is often the bottleneck (Kimi K2 [5]: **>10,000 concurrent sandboxes**).
- **Solution. AgentCore Runtime is that sandbox** — one isolated microVM session per episode, **in your own AWS account** (IAM/SigV4, VPC), bring-your-own container, CPU by default and **GPU on Instances** — wired into the **slime** trainer through **one hook**; the agent loop runs **in the episode's session** (as in DeepSeek DSec) and scales with the sandbox, not the trainer.
- **Proven.** The full loop runs **end-to-end on real hardware** across **4 main demos + 1 side note + 1 experimental** (code, math, SQL, RAG, tool use, data science, terminal tasks, CUDA kernels), each smoke a standard **10-step run** with real rewards from an in-sandbox checker. **Most literature tasks need no custom image**: one generic stdlib Runtime serves them all.
- **How you adopt it.** Start on **one generic-image Runtime** → seed a task → train via one `generate` hook; build a task-specific image only when the task needs its own OS, a $>$ 2 GB toolchain or a GPU (slide 69).

Source: Kimi K2 tech report [5] (slide 72); project demo smokes 2026-09-18 to 2026-09-28 (Section 5); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Frontier labs score sandboxed output with programmatic verifiers — Kimi K2 runs >10,000 concurrent sandboxes

Untrusted output runs in an isolated sandbox; a **programmatic verifier** produces the reward — the dominant recipe in recent tech reports.

Table 1 **What frontier models verify, and how**

Model	Verifier	Tasks	Reward
DeepSeek-R1 [2]	rule verifier + compiler	math, competitive code	tests pass / match
Qwen3 [3]	RLVR + live tool env	math, code, tool-use, RAG	rule + judge
Llama-Nemotron [4]	LLM-judge	science, math, instruction	match + format
Kimi K2 [5]	simulator + real exec sandboxes	agentic tool use, code, SWE	tests + checks

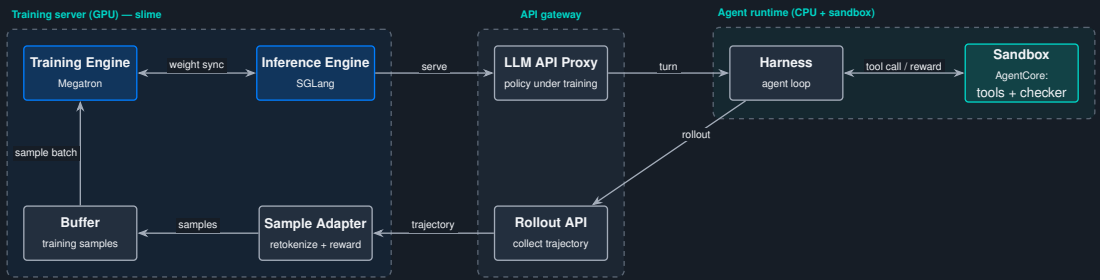
- **Two verifier styles:** *verifiable / rule-based* (binary, ungameable) vs *reference-based judge* (open-ended). This project uses **RLVR only**.
- Same recipe throughout this deck: **one isolated sandbox per episode**, a **programmatic checker** for the reward.

Source: Tech reports [2]–[5], see References (slide 72); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore fills the sandbox seam of a disaggregated agentic-RL architecture

Figure 1 Reference architecture: generation lane (top, →), training-data lane (bottom, ←)



Disaggregated design after Agent Lightning [1], slime [9] as trainer. **AgentCore fills the sandbox seam.**

© 2026, Amazon Web Services, Inc. or its affiliates.

Five components with one job each, joined by a generation lane and a training-data lane

The pipeline — one job each

- **Trainer** / **Inference** (GPU) — learn, and generate turns at the *current* policy (kept fresh by **weight sync**).
- **Proxy** (CPU) — makes every model call hit the *policy under training*, not an external API.
- **Harness** (CPU) — the agent loop that owns one episode end-to-end; it runs **in the sandbox session**, dispatched by a trainer-side tier of `W` workers.
- **Sandbox** (CPU/GPU) — the isolated box untrusted output runs in; the **checker scores the reward**.

Two lanes, one cycle

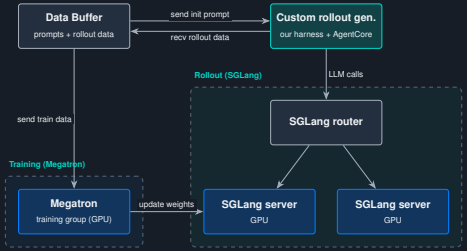
- **Generation lane (→):** trainer weights → SGLang → proxy → harness turn → sandbox tool call → observation → next turn.
- **Training-data lane (←):** trajectory → reward attached → loss-masked sample → buffer → trainer step → weight sync.

In plain English: AgentCore fills the sandbox seam — ad-hoc + isolated per episode. The harness must scale **with** inference and the sandbox, so it runs **in the episode's session** (one loop per sandbox), not in the trainer.

© 2026, Amazon Web Services, Inc. or its affiliates.

slime’s fully-async rollout keeps the trainer fed while long, variable AgentCore episodes run

Figure 2 **slime: Megatron + SGLang bridged by a Data Buffer**



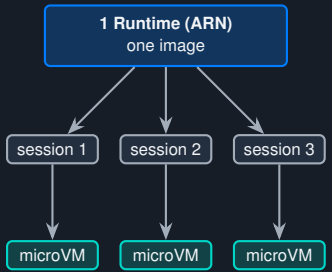
slime [9] = Megatron (training) + SGLang [27] (rollout), bridged by a **Data Buffer**; the agentic loop plugs into its rollout **generation hook** — no fork of the training kernel.

- **Sync mode** alternates train / rollout — each side’s GPUs idle while the other runs.
- **Async driver** (`train_async.py`) overlaps them: Megatron trains batch *k* while the rollout side already generates batch *k+1*.
- **Fully-async rollout** (`fully_async_rollout`) keeps a *fixed pool of in-flight episodes across rollout boundaries*, so a step never waits on the slowest trajectory — the **long-tail** problem of agentic RL.
- **Why it matters here:** AgentCore sandboxes have **variable, sometimes long** per-episode latency; async rollout keeps the trainer fed while they run — at the cost of **staleness** (rollouts generated on an older policy version). This project runs the fully-async path (`MODE=B`).

© 2026, Amazon Web Services, Inc. or its affiliates.

One Runtime serves many sessions, each on exactly one microVM — Runtimes track images, not tasks

- **Runtime** — a versioned, containerized host behind an ARN (`CreateAgentRuntime` [D9]). The deployable unit; holds no per-user state.
- **Session** — one episode, addressed by a `runtimeSessionId`. First invoke cold-starts compute; the same id re-routes to that warm compute [D3].
- **microVM** — the unit of isolation: own kernel, CPU, memory, filesystem; sanitized on termination [D2].



Cardinality is 1 : N : 1 — one Runtime serves many sessions; each session = exactly one microVM. **The number of Runtimes tracks distinct sandbox *images*, not tasks.**

Source: AgentCore docs [D2], [D3], [D9] (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

N concurrent episodes run as N session ids on ONE Runtime — concurrency means more sessions, not more Runtimes

- The `generate` hook opens **one session per episode** (`CONC` in flight at once); each session id pins that episode to **its own microVM** (or its own GPU instance).
- Every tool op of an episode carries the *same* `runtimeSessionId`, so it returns to the **same warm microVM** — state persists within the episode, isolation holds across episodes.
- The capacity provider places those `N` sessions for you; **one image (one ARN)** backs them all. Add a **second ARN** only for a different image/substrate (e.g. GPU), then round-robin the pool.

In plain English: to run more episodes at once you open more sessions on the same Runtime; you add a Runtime only when you need a different image.

© 2026, Amazon Web Services, Inc. or its affiliates.

A microVM gives each untrusted episode its own kernel, which a Docker container does not

Running **untrusted, model-generated code** — isolation is the whole point: one episode's code can't see another's files, the trainer's weights, or the host, and the box is wiped at session end.

Table 2 **Isolation options compared**

	Docker container	AgentCore microVM	E2B
Boundary	shared host kernel	**own kernel** (Firecracker-class)	own kernel
Runs your image?	yes	**your OCI image in a microVM**	Dockerfile templates
Managed by	you	**AWS, in your account** (IAM/VPC)	E2B service
GPU	host-dependent	**via Instances**	not core

© 2026, Amazon Web Services, Inc. or its affiliates.

CPU work belongs on the microVM; Instances exist only to add a GPU, one whole instance per session

Table 3 **Two compute types [D5]**

Property	Serverless microVM	Instances (GPU)
GPU	none	g4dn/g5/g6/g6e… + inf2
Max session	8 h	14 d
Isolation	1 microVM / session	1 instance / session (**1 GPU**)
Filesystem	ephemeral	persistent EBS
Agents / session	1	many

- Instances = EC2 in your account via a capacity provider [D5]. **No cross-session GPU time-slicing** — scale by adding sessions/instances, not by fractioning one (no MIG on L4).
- **One session = one whole instance** (session→instance is 1:1 — you get the *instance*, not a GPU). Two ids acquire two *separate* instances, so you can't split one multi-GPU box across sessions.
- To use every GPU of one instance: **one shared session + a self-managed in-instance Ray farm** (`demo_gpu_farm`, `FARM=fractional|whole`; trusted-only; slide 50).

Source: AgentCore Instances docs [D5], Runtime docs [D3] (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Three microVM limits shape every image: a ∼2 GB image cap, ephemeral disk, metered vCPU/RAM

Serverless isolation is cheap, but the microVM is **small and ephemeral**.

Table 4 **The microVM's resource envelope**

Limit	Value	Why it bites	Work-around
Image size	≤ ∼**2 GB** (arm64, project-measured)	a CUDA/torch wheel alone is ∼2.7 GB → build fails	install heavy wheels **at** `CMD`, not baked (GPU image: ∼90 s cold)
Disk	**ephemeral only** — no persistent volume [D3]	a `git cloned` repo / installed pkg is **wiped at session end**	durable state → S3 / session Memory, or move to **Instances** (EBS) [D5]
vCPU / RAM	**metered, not sized** (vCPU-hr + GB-hr) [D13] — no fixed knob	a data-heavy checker can exceed the account quota	check **Service Quotas**; big fixed footprint → **Instances** (EC2 spec)

- **Example:** the g6 sandbox image can't bake torch (>2 GB) → `pip install torch` at `CMD` so the wheel lands in the ephemeral workspace at boot; a big SWE repo + persistent workspace → move to **Instances** (EBS, 14 d).
- **Rule of thumb:** *files-in-a-session* for light deps → *`CMD`-time install* for medium → *Instances* when the footprint is large and **fixed**.

Source: Image cap measured by the project (arm64 builds); AgentCore docs [D3], [D5], [D13] (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every Runtime meets one contract — an ARM64 image serving :8080 with /invocations and /ping

Contract [D17]: ARM64 image, serve `0.0.0.0:8080` with `/invocations` + `/ping`. Three ways to put your code on a Runtime (increasing control): the `@aws/agentcore` CLI [D16] (zero-Docker zip) → the `bedrock-agentcore` SDK [D15] (`@app.entrypoint` implements the contract) → **bring-your-own container** (max control — this project).

Dockerfile (CPU sandbox)

```
FROM public.ecr.aws/docker/library/python:3.12-slim # CPU sandbox
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
CMD ["python","-m","agentcore_slime_demo.sandbox_server_agent"] # serves :8080
```

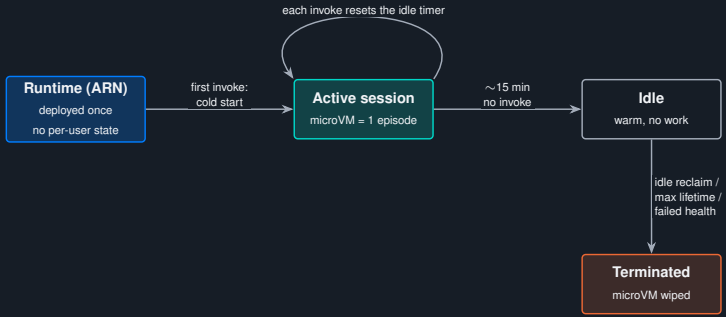
- **GPU image:** point `LD_LIBRARY_PATH=/usr/lib64` at the injected `libcuda.so.1`, and `pip install torch` **at** `CMD` (not baked) to stay under the **2048 MB** cap.
- **Deploy:** ECR create → `buildx --platform` (arm64 microVM / amd64 GPU) → push → `CreateAgentRuntime` → (GPU) `CreateCapacityProvider` (`g6.xlarge`, VPC, `idleInstanceTimeout=600`) + attach.

Source: Runtime service contract [D17], SDK [D15], CLI [D16] (slide 74); project deploy scripts; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Session launch and termination are both implicit; each invoke resets the idle timer

Figure 3 Session lifecycle



Launch is **implicit** (first invoke); terminate is **implicit** (idle / max-lifetime / health). A heartbeat ping resets the timer (`HEARTBEAT=1`, any demo; slide 45).

Source: AgentCore Runtime docs [D3] (idle 15 min) (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Concurrency multiplies instances, not data — only CONC episodes are ever in flight

- **Launch = the first `invoke_agent_runtime` on a fresh `runtimeSessionId`.** There is no explicit "start" call; the id ($\geq$33 chars) *is* the launch. The first op cold-starts the microVM (or GPU instance — minutes on a cold GPU); every later op with that id re-routes to the same warm box.
- **Terminate = passive.** No explicit stop on the happy path — the session ends on **idle reclaim ($\sim$15 min)**, **max lifetime** (8 h microVM / 14 d instance), or **failed health**. Each invoke resets the idle timer.
- **Consequence in the RL loop:** a **100-row dataset does not launch 100 instances** — only `CONC` episodes are in flight, recycling through rows over steps. **Concurrency multiplies instances, not data.**
- The **weight-sync pause can trip the idle clock** and silently lose a warm session $\rightarrow$ the resilience levers in §6 (slide 68).
- **Latency: measure, don't assume** — every op is instrumented (`sandbox_ms`, `sglang_gen_ms`).

Source: AgentCore docs [D3], [D5], [D8] (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore gives SaaS-level “no fleet to run” without your code or data leaving your account

Every agentic-RL team must solve “run untrusted code, isolated, at scale.” The options and the trade:

Table 5 **Rolling your own sandbox vs SaaS vs AgentCore**

	Self-managed K8s / Firecracker	**E2B / Modal (SaaS)**	**AgentCore Runtime**
Isolation	microVM (you operate)	microVM (their cloud)	**microVM, per session**
In your account / VPC	yes — you build it	**no** — their cloud	**yes** — IAM/SigV4, VPC
Bring-your-own container	yes	limited	**yes** (OCI image)
GPU sandbox	you wire it	limited / none	**yes, on Instances**
Ops burden	**high** (run the fleet)	low	**low** (managed)
Session length	you choose	provider cap	8 h μVM / **14 d Instance**

The client point: AgentCore gives SaaS-level “no fleet to run” **without** your code or data leaving your account.

Source: AgentCore docs [D2], [D3], [D5] (slide 74); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Running untrusted output is safe by construction: per-session microVM, your account, your image, ephemeral

Every episode executes model-generated code you did not write. AgentCore makes that safe **by construction**:

- **Per-session microVM isolation** — a hardware-virtualized boundary (not a shared container); one episode's code cannot see another's.
- **In your account, in your VPC** — the sandbox runs under **your IAM**, authenticated with **SigV4**; VPC-mode Runtimes keep traffic on **private endpoints** — **nothing leaves your account**.
- **Bring-your-own image** — you own every dependency, user, and filesystem, and control what the sandbox can reach on the network.
- **Ephemeral by default** — a fresh session per episode, reclaimed on idle; no state bleeds across rollouts unless you deliberately snapshot it (`demo_gpu_farm SNAPSHOT_EVERY=1`).

In plain English: this is the enterprise reason to run agentic RL on AWS rather than a third-party sandbox cloud.

© 2026, Amazon Web Services, Inc. or its affiliates.

CPU sandboxes are cheap; GPU cost concentrates, and that is where the Ray farm's packing pays

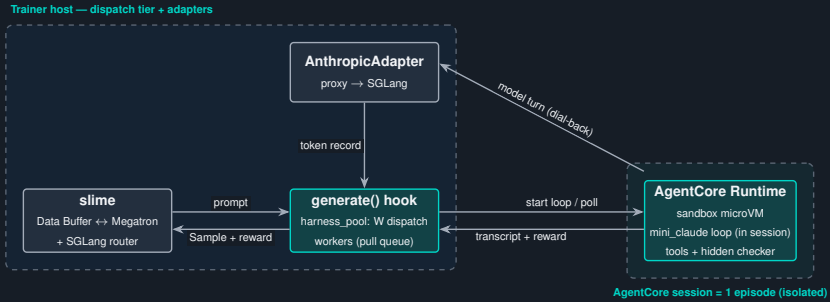
- **Billing model:** you pay for sandbox **vCPU-seconds + GB-hours** while a session is live [D13], plus **Instance** hours for GPU sandboxes — not a flat per-sandbox fee. Idle sessions are reclaimed ($\sim$15 min), so **warm-but-unused time is bounded**.
- **CPU sandboxes are cheap; the bottleneck is elsewhere:** for cheap-checker families (math, code, SQL, tool-use) the measured sandbox op is **$\sim$0.1 s** (p50 123 ms / p95 900 ms) — generation dominates, so you scale inference, not sandboxes.
- **GPU is where cost concentrates — and where packing pays:** KernelBench is sandbox-bound (`exec` p50 **8.5 s** $\gg$ generation 2.5 s). Isolated needs **N whole g6 at once**; the Ray farm (`demo_gpu_farm`, `FARM=fractional`) packs **N episodes onto one GPU** (`num_gpus=0.5` $\rightarrow 2\times$), so you provision **one** instance, not N.
- **Primary lever is capacity, not just cost:** on a capacity-starved GPU fleet you often *cannot* launch N instances at all — the farm makes the run *possible*, and cheaper.

Source: AgentCore observability docs [D13]; project `[sbx-op]` latency records from the CPU smokes and the `demo_gpu_farm` `LOCAL_GPU=0` committed run (slide 53); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

One generate() hook and one runtimeSessionId per episode connect slime to AgentCore

Figure 4 The call path: trainer host (dispatch tier + adapters) and one isolated AgentCore session per episode



One `generate()` hook, **one** `runtimeSessionId` **per episode**; the agent loop and its tools run in that session; each trainer→session op = one SigV4 `invoke_agent_runtime`.

© 2026, Amazon Web Services, Inc. or its affiliates.

slime's custom generation function lets the agentic workload plug in with no fork of the trainer

slime exposes the rollout as a **custom generation function**, so the agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path ...generate.generate` — hands the episode to the **harness tier** (`harness_pool.py`, default in every demo): a free dispatch worker takes a sandbox session, then the least-loaded SGLang engine, and starts the agent loop **inside** that session (`runtimeSessionId`).
- **Design choice:** the harness scales **with the sandbox, not the trainer** (DSec: move the agent loop out of the trainer). The in-session `mini_claude` loop dials the engine's `AnthropicAdapter` back; the trainer keeps only dispatch + adapters. (`demo_cpu_branching` runs the real Claude Code CLI in its microVM instead.)
- The returned **loss-masked** `Sample` **+ reward** flows straight back into slime's buffer — no fork of Megatron/SGLang.

© 2026, Amazon Web Services, Inc. or its affiliates.

slime's training kernel owns neither the Runtime nor the session

Runtime (create/delete the ARN)

The **deploy driver**, once before training. slime just gets the ARN.

Session (per episode)

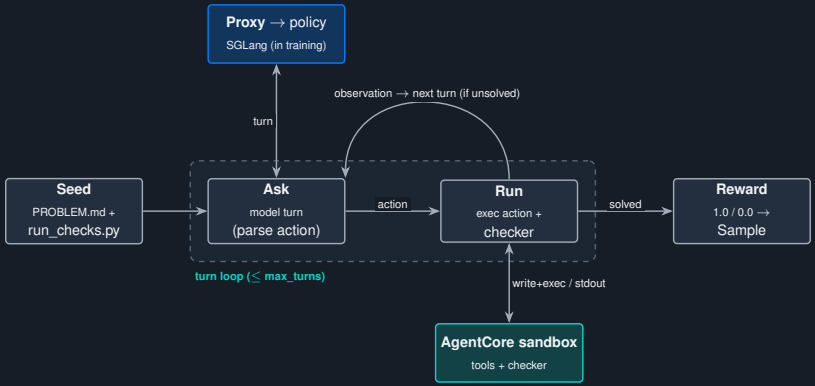
The `generate` hook's sandbox client: **launch implicit** (first invoke cold-starts the microVM); **terminate explicit** (`StopRuntimeSession` when the episode returns; idle reclaim is the backstop).

Because an idle session is reclaimed *silently*, the weight-sync pause can trip the idle clock and lose a warm session — the failure mode the `HEARTBEAT=1` knob (any demo on an own Runtime) addresses (slide 45).

© 2026, Amazon Web Services, Inc. or its affiliates.

Each episode loops Seed, Ask, Run, Reward — an unsolved turn feeds its observation back to the next

Figure 5 The episode, end to end



Seed → Ask (proxy) → Run (sandbox + checker) → Reward; unsolved loops back.

© 2026, Amazon Web Services, Inc. or its affiliates.

One MBPP episode runs in six steps, from a once-only deploy to the weight sync

1. **Deploy (once):** build the arm64 sandbox image → `CreateAgentRuntime` → get the ARN (the deploy driver, not slime).
2. **Launch run:** `run.sh` sets `SANDBOX=agentcore-runtime`, `DATASET`, `CONC`, `HARNESS_WORKERS` (default RB × NS), then `ray job submit ... train_async.py --custom-generate-function-path ...generate.generate`.
3. **Dispatch + open session:** slime hands a prompt to `generate()`; a free harness worker pulls it, opens a **fresh session** (cold-starts that episode's microVM, `runtimeSessionId`), then takes the **least-loaded SGLang engine** (sticky for all turns).
4. **Seed + start the loop:** `write_file PROBLEM.md` + the stdlib `session_runner.py`; `exec` starts the `mini_claude` loop **inside the session**.
5. **Turn loop (in the session):** each model turn dials back to the engine's `AnthropicAdapter` (the **proxy**, TITO tokens); tools (`write_file` / `bash` / `read_file`) run locally in the microVM; the trainer only long-polls.
6. **Reward & learn:** loop killed → the **hidden checker** is written and run → exit code = **reward 1.0/0.0** → the adapter's token record → one loss-masked `Sample` → buffer → Megatron step → **weight sync** to SGLang.

© 2026, Amazon Web Services, Inc. or its affiliates.

The generate hook takes a sandbox first, then an engine, and returns one bare loss-masked Sample

slime calls this once per prompt: a free dispatch worker takes **one sandbox per episode**, then an engine, and runs the loop **in that session**; one loss-masked `Sample` comes back.

```
# agentcore_slime_demo/generate.py (--custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    task, pool = _task_from_sample(sample), harness_pool.get_pool(args)
    async def episode(worker): # W = HARNESS_WORKERS, pull queue
        async with pool.sandbox(key, open_session) as (sb, wait_ms, slot): # sandbox FIRST
            with pool.engine() as (e, engine): # least-loaded engine, sticky
                return await mini_claude_episode.run(args, sample, task,
                                                     sampling_params, sb, engine, ...)
    return await pool.run(episode) # ONE bare Sample
```

- Loss mask = **1 on the policy's own tokens, 0 on prompt / observation / tool tokens**.
- The adapter keeps the **recorded** `token_ids` **per turn** — no re-tokenization (TITO, slide 62); a *bare* `Sample` (slime's fully-async path chokes on nested lists).
- No in-trainer fallback: a session that can't host the loop raises `HarnessPlacementError`.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every trainer-to-session op is one SigV4 invoke, and switching CPU to GPU is one env var

Each trainer→session `write_file` / `exec` / `read_file` (seed, start/poll the loop, grade) = **one SigV4** `invoke_agent_runtime` [D8] keyed by the episode's session id; boto3 offloaded with `asyncio.to_thread` so `N` episodes run concurrently (with retries for transient blips + the GPU cold launch).

Client (`agentcore_runtime_sandbox.py`)

```
# client -- agentcore_runtime_sandbox.py
def _invoke_sync(self, payload):
    resp = self._client.invoke_agent_runtime(
        agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id, # >=33 chars; one per episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())
```

Server (deployed image)

```
# server (deployed image) -- the checker runs HERE,
# so the microVM is the reward source
app = BedrockAgentCoreApp()
@app.entrypoint
def entrypoint(request): # dispatch on op: ping / write_file / read_file /
        exec
    return handle(request)
app.run() # serves 0.0.0.0:8080 (/invocations + /ping)
```

CPU → GPU is one env var (`SANDBOX=`) — zero change to the loop.

Source: InvokeAgentRuntime API [D8], Python SDK [D15] (slide 74); project code; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Task diversity is absorbed inside one generic image; only a real substrate difference needs its own ARN

One microVM = one image per session. Task diversity is absorbed *inside* one generic image, in increasing order of how structural the difference is:

1. **Files, not images** — most CPU families (math `sympy`, code `unittest`, SQL `sqlite3`, tool-use, light CTF) share **one generic Python image**; prompt, seed files, and hidden checker are written into *that episode's session*.
2. **Per-session install** — heavier deps go into the ephemeral workspace: `CMD`**-time** (the GPU image installs the ∼2.7 GB torch wheel at boot to stay under the ∼2 GB arm64 cap) or **seed-time** `git clone`/`pip`.
3. **Per-capability image** — only a genuine substrate difference (chiefly the **GPU** toolchain: `nvcc` + a real GPU, or a prebuilt dependency set) needs its **own Runtime ARN**, routed by `task.image` (`demo_cpu_terminalbench` at one image per task, the GPU demos). Most CPU families need none: one generic image carries pandas / scipy / sklearn.

Source: Project image builds (torch wheel size, arm64 image cap); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each demo stands in for a real agent a client would train — find your workload here

Table 6 **Workload → demo**

Your workload	Demo
Coding / math / SQL / RAG / tool use / data science	`demo_cpu_tasks` (`DATASET=mbpp`, `humaneval`, `gsm8k`, `math500`, `deepscaler`, `gretelsql`, `hotpotqa`, `xlam`, `mcp`, `libs`, `mixed`) — on **one generic stdlib Runtime**, no custom image
Terminal / DevOps agents that act on the whole OS	`demo_cpu_terminalbench` (one image per task)
SWE agents on real repos	`demo_cpu_swebench` (SWE-bench Verified, one image + Runtime per row on m5 Instances)
A high-throughput rollout fleet	`demo_cpu_tasks SCALE=1` (4 engines : 256 harness workers : 256 microVMs; the harness tier is the default in every demo)
GPU kernels / systems	`demo_gpu_farm`: one warm GPU instance serving many sessions, on this box's GPUs 6,7 (`LOCAL_GPU=1`, default) or a remote g6 (`LOCAL_GPU=0`); `FARM=fractional`|`whole`; trusted-only
A real coding-agent CLI as the harness	`demo_cpu_branching` (experimental)
Side note: a trusted checker with a $>$ 2 GB toolchain	`demo_cpu_instance_pool` (warm m5 pool, Landlock jail, not full isolation)

© 2026, Amazon Web Services, Inc. or its affiliates.

Seven chapters walk from the tasks to GPU sandboxes and an experimental real CLI

4 demos + 1 side note + 1 experimental, 7 chapters

1. **The tasks** — dataset, URL, env deps, image or not (slide 33).
2. **Most tasks need no custom image** — one generic stdlib Runtime, harness in the sandbox session (`demo_cpu_tasks`; slide 36).
3. **Custom images at scale** — one image + Runtime per task (`demo_cpu_terminalbench`), and per dataset row on m5 Instances (`demo_cpu_swebench`; slide 40).
4. **Hiding the container launch** — pre-open + heartbeat (`warmstart/` probe; slide 42).
5. **Harness scaling** — the decoupled harness tier (default in every demo) scaled to 4 : 256 : 256 (`demo_cpu_tasks SCALE=1`; slide 46).
6. **GPU sandboxes** — ONE GPU instance serving MANY sessions (`demo_gpu_farm`): local farm (`LOCAL_GPU=1`) or remote g6 (`LOCAL_GPU=0`, formerly `demo_gpu_single`), `FARM=fractional|whole`, trusted-only (slide 49).
7. **Experimental** — one episode, many samples, real Claude Code CLI (`demo_cpu_branching`; slide 58).

Side note — a warm m5 pool shared by `sub_sessions` (`demo_cpu_instance_pool`; slide 55): our pooling on AgentCore Instances, not full isolation.

In plain English: policy **Qwen3-4B**, trainer **slime**, harness **in the sandbox session** (the decoupled harness tier, `harness_pool.py`, in every demo); the same six step scripts everywhere. Every run is a **smoke** (10 train steps · 1 eval) — not convergence.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pick the lowest sandbox tier that works — a tier-1 task scores the same on tier 2

Pick the lowest tier that works. A tier-1 task scores the same on tier 2 (`compare_backends.py`).

Table 7 Which sandbox does a task need? Three tiers

Tier	Use it when	Here	Demos
1 · Code Interpreter (`SANDBOX=agentcore`), no image	Python 3.12 + sci stack, sqlite, gcc, node is enough; no git / internet / GPU	every literature task *scores* here (probe), but it **can't host the in-session loop** (no route back) → no demo uses it now	— (historical `demo_cpu_tasks` smokes)
2 · own Runtime, generic image (`SANDBOX=agentcore-runtime`)	the task needs no special env; the loop runs in the session, heartbeat past idle, VPC	every literature task; DeepScaleR at 256 sessions (`SCALE=1`); heartbeat	`demo_cpu_tasks` (default), `warmstart/`
3 · own Runtime, custom image	the **environment** differs	own OS per task; SWE repo per row (m5); node + CLI; 4 GB stack (m5 Instance); CUDA (g6 Instance)	`...terminalbench`, `...swebench`, `...branching`, `demo_gpu_farm`; side note `...instance_pool`

Source: Code Interpreter environment probe 2026-09-25; `compare_backends.py`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

All five CPU demos ran end to end on AWS; SWE-bench still scores reward 0

Table 8 **All demos at a glance (1/2) — CPU**

Ch.	Demo	Sandbox substrate	Task	Status
1–2	`demo_cpu_tasks`	own **generic-image** μVM Runtime, fresh session per episode	any literature RLVR dataset, or a mix	**passed** on CI (2026-09-25, before the switch): gsm8k **0.90**, libs **1.0**
3–4	`demo_cpu_terminalbench`	μVM, one image + Runtime **per task** (218)	Terminal-Bench	**passed** (2026-09-23): boot hidden, 4/382 solved
3	`demo_cpu_swebench`	**m5 Instance**, one image + Runtime **per row** (20)	SWE-bench Verified	deployed 20/20; oracle 1/20 usable; smoke 24 episodes, 0 infra errors, reward 0
5	`demo_cpu_tasks SCALE=1`	256 fresh μVM sessions in flight	DeepScaleR	256-worker RL smoke (2026-09-28): 592 episodes, eval 0.27; pool check ok (old $4\times4\times4$ lanes: eval 0.358)
side note	`demo_cpu_instance_pool`	warm **m5** pool, Landlock jail per exec	DeepScaleR, $>$ 2 GB image	RL smoke **passed** 2026-09-25, eval 0.7875, 2 cold boots

Knobs, not demos: `HEARTBEAT=1` and `HARNESS_WORKERS` (every demo), `SCALE=1` (`demo_cpu_tasks`, the harness-scaling run), `compare_backends.py` (`demo_cpu_tasks`), `SNAPSHOT_EVERY=1` (`demo_gpu_farm`), and the decided `SLIME_CUDA_VISIBLE` / `SLIME_NUM_GPUS` pin (unset = 8-GPU placement).

Source: Demo smoke runs 2026-09-23, 2026-09-25, 2026-09-28 (details on the per-demo slides); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every GPU farm site and mode reached 0.65 or better; the remote-g6 site's op=open 500s are unexplained

Table 9 **All demos at a glance (2/2) — GPU and experimental**

Ch.	Demo	Sandbox substrate	Task	Status
6	`demo_gpu_farm` · `LOCAL_GPU=0`	remote warm `g6.xlarge` (L4), one shared session (was `demo_gpu_single`)	KernelBench	**trained** (0.65); [PENDING: root cause of the op=open 500s]
6	`demo_gpu_farm` · `FARM=fractional`	ONE GPU instance, many sessions: 0.5 + 0.5 of **one** GPU (trusted-only)	KernelBench	**passed** (2026-09-25): eval **0.65**
6	`demo_gpu_farm` · `FARM=whole`	ONE GPU instance, many sessions: one **whole** GPU each (trusted-only)	KernelBench	passed (2026-09-21): 0.65 / 0.70
7	`demo_cpu_branching` *(exp.)*	μVM with node + `claude`	SWE bug-fix, real Claude Code CLI	runs end to end: fan-out p50 4, eval 0.47; advantage bug fixed, [PENDING: branching re-run with the advantage fix]

Source: Demo smoke runs 2026-09-21, 2026-09-25 (details on the per-demo slides); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 1: the code and math datasets need only the stdlib — no custom image

Every task is a real RLVR dataset, scored by a programmatic checker. The question: does it need a custom image?

Table 10 The tasks (1/3): code and math

DATASET	Dataset (URL)	Train / eval	Env deps	Image?
mbpp	google-research-datasets/mbpp	500 / 8	stdlib	no
humaneval	openai/openai_humaneval	140 / 24	stdlib	no
gsm8k	openai/gsm8k	200 / 40	stdlib	no
math500	HuggingFaceH4/MATH-500	160 / 40	stdlib	no
deepscaler	agentica-org/DeepScaleR-Preview-Dataset	400 / 40	stdlib	no

Source: Hugging Face dataset cards (linked); project data-prep splits (`build_real_datasets.py`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 1: the SQL, RAG, tool-use and data-science datasets also need no custom image

Table 11 The tasks (2/3): SQL, RAG, tools, data science

DATASET	Dataset (URL)	Train / eval	Env deps	Image?
gretelsql	gretelai/synthetic_text_to_sql	300 / 40	sqlite3	no
hotpotqa	hotpotqa/hotpot_qa (distractor)	300 / 40	10 passage files	no
xlam	minpeter/xlam-function-calling-60k-parsed	300 / 40	stdlib	no
mcp	Team-ACE/ToolACE (MCP-style)	300 / 40	stdlib	no
libs	synthetic datasci + scipy + sklearn	72 / 6	pandas, scipy, sklearn	**no** — preinstalled
mixed	synthetic math + SQL + CTF + datasci	174 / 9	stdlib + pandas	no

Source: Hugging Face dataset cards (linked); project data-prep splits; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 1: a custom image is needed only for a pinned OS, >2 GB, CUDA or a missing toolchain

The pattern: literature datasets need only Python + its scientific stack → the **default sandbox**. A custom image only for a **pinned OS**, > **2 GB**, **CUDA**, or a missing toolchain (CI: **no git, no internet**).

Table 12 **The tasks (3/3): where a custom image IS needed**

Task (chapter)	Dataset (URL)	Train / eval	Env deps	Image?
Terminal-Bench (Ch. 3)	laude-institute/terminal-bench	135 / 31 usable	a **pinned OS per task**	**one per task**
DeepScaleR, heavy stack (side note)	DeepScaleR (previous table)	400 / 40	torch + sci stack, > **2 GB**	**CPU Instance**
KernelBench (Ch. 6)	ScalingIntelligence/KernelBench	100 L1 / 20 L2	CUDA + `nvcc` + torch	**GPU Instance**
SWE bug-fix (Ch. 7)	synthetic SWE tasks	64 / —	node 22 + `claude` CLI	yes

Source: Dataset repos (linked); Terminal-Bench oracle validation 2026-09-23; Code Interpreter probe 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 2: demo_cpu_tasks shows that one generic stdlib image covers every literature dataset

Use case covered.
RLVR on the literature's datasets — code, math, SQL, RAG, tool calling, data science — alone or mixed.

Why this demo.
It answers *do I have to build a sandbox image per task?* Mostly **no**: one generic stdlib image has the deps (the Code Interpreter probe proved it). One demo, not one per dataset: same harness, sandbox, trainer. `SCALE=1` is also the harness-scaling run (Ch. 5).

Design considerations.

- **Own generic-image Runtime** (VPC) by default, the harness **inside the sandbox session**; the Code Interpreter was dropped (it can't dial the adapter back).
- **Harness tier** (every demo): `HARNESS_WORKERS` (default RB × NS), fresh session per episode, least-loaded engine; `SANDBOX_SESSIONS=M` (opt-in) pools M sessions per image with a fresh workdir.
- **One harness**: self-describing rows rebuild each task; mixes are GRPO-safe (per-group advantage).
- **Heartbeat on by default** (own Runtime), idle timeout 3600 s.
- **Rejected:** a per-library image pool — one image with pandas / scipy / sklearn covers them.

Source: `demo_cpu_tasks` configuration (`run.sh`, `demo.env`); Code Interpreter probe 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 2: on the Code Interpreter the invoke rate binds first, at ∼75 episodes in flight

- **Per episode (CI, before the switch):** start session → `write_file` / `exec` / `read_file` → stop; its own microVM, no ARN, no IAM role.
- **Env (probe 2026-09-25):** Python 3.12, numpy, pandas, scipy, sklearn, sympy, torch-CPU, sqlite3, gcc, node — **no git, no internet**. All **13 families** score 1.0.
- **The invoke rate binds first:** ∼8 ops per 20 s episode → **∼75 episodes in flight**; raise it before `CONC` $\gtrsim$ 32.

Table 13 **Code Interpreter quotas vs this demo's use**

Quota (us-west-2 default)	Code	Default	This demo
Concurrent CI sessions	L-CAF6F552	1000	`CONC` × engines = **8**
`StartCodeInterpreterSession`	L-6B09F374	30/s	∼1 per episode
`InvokeCodeInterpreter`	L-04A719E9	30/s	∼6–10 per episode

Source: AWS Service Quotas, us-west-2 defaults; Code Interpreter probe 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 2: gsm8k scored 0.90 and libs 1.0 on the Code Interpreter — a plumbing proof, not a learning curve

- `gsm8k` — **PASSED:** `eval/gsm8k` **0.90**; 425 CI sessions, **0 infra errors**; step ∼42–54 s.
- `libs` — **PASSED, no image:** `eval/libs` **1.0**, `raw_reward` 1.0 on 9/10 steps; datasci 132 / scipy 109 / sklearn 108 episodes; **1883 ops, all ok**.

In plain English: both near-saturated → a **plumbing proof, not a learning curve**. Session open (∼1.8 s) is the largest sandbox cost.

Table 14 **Sandbox op latency,** `libs` **run**

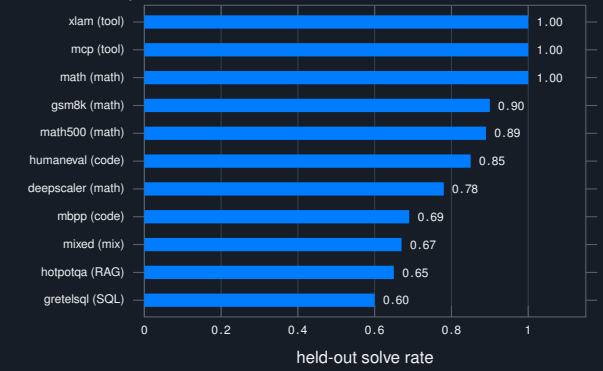
op (libs run)	count	p50	p95
`start_session`	350	**1.83 s**	3.50 s
`write_file`	834	69 ms	127 ms
`exec` (imports pandas/scipy/sklearn)	353	1.34 s	2.10 s
`stop_session`	346	150 ms	1.34 s

Source: `demo_cpu_tasks` Code Interpreter smokes (`gsm8k`, `libs`), us-west-2, 2026-09-25; `[sbx-op]` records; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 2: eleven own-Runtime smokes solved 0.60–1.00 held-out, all of them generation-bound

Figure 6 Held-out solve rate, 10-step smokes (own Runtime, 2026-09-18→19)



Source: `demo_cpu_tasks` own-Runtime 10-step smokes, 2026-09-18 to 2026-09-19; checked 2026-09-28.

- Checker and reward are identical on both backends (`compare_backends.py` asserts it).
- **Code:** `mbpp` 0.69 · `humaneval` 0.85
- **Math:** `math` 1.00 · `gsm8k` 0.90 · `math500` 0.89 · `deepscaler` 0.78
- **SQL / RAG:** `gretelsql` 0.60 · `hotpotqa` 0.65
- **Tool use:** `xlam` 1.00 · `mcp` 1.00; **Mix:** `mixed` 0.67
- Re-running them on the Code Interpreter: [PENDING: CI smokes for the other DATASET values].

In plain English: cheap checkers (∼0.1 s sandbox op vs ∼1.8 s decode) make all of these **generation-bound**: scale inference, not sandboxes.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 3: demo_cpu_terminalbench runs hundreds of custom images with one image and Runtime per task

Use case covered.
Agentic terminal and SWE-style tasks where every task pins its own OS image — Terminal-Bench's ∼229 containerized tasks [6], graded by each task's hidden tests.

Why this demo.
The other side of Ch. 2: when a task *does* need a custom image, how do you run **hundreds**? **One image + one Runtime per task**, strict per-task routing. Its own demo: per-task images, a fresh microVM per episode and a tool-calling loop all differ from `demo_cpu_tasks`.

Design considerations.

- **A fresh microVM per episode, never reused** — the tasks mutate the OS.
- **Oracle validation before training** — drops the 47 tasks whose reference fails here.
- **Look-ahead pre-open** hides the cold start (Ch. 4, slide 43); **heartbeat always on**.
- **Single-container tasks only** (one image ↔ one sandbox).
- **Quota:** pre-opened + running sessions vs Active Session Workloads (5000).

Source: Terminal-Bench [6]; `demo_cpu_terminalbench` deploy + oracle check, 2026-09-23; AgentCore quota (Active Session Workloads); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 3: the Terminal-Bench smoke hid the cold start fully, but ∼95% of booted time idled and 4/382 solved

- **The pool:** one arm64 image (ECR `acs-tb-*`) + one Runtime (`acs_tb_*`) per task → `tasks/arn_by_image.json`. **218** / **229** deployed; oracle check (untouched env fails, reference passes) → **166 usable** = 135 train / 31 eval.
- **Strict routing:** `task.image` → `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`; an unmapped task is an **error**, never a fallback Runtime.
- **The loop:** the tool-calling `mini_claude` loop **inside the episode's session** (SGLang `qwen25` tool parser), dispatched by the shared harness tier (`HARNESS_WORKERS` = RB × NS); reward = the task's hidden oracle, run by `grade.sh` from `/opt/acs/oracle`, outside the workdir.

Smoke — PASSED (2026-09-23): 382 episodes in ∼70 min, 0 infra errors.

- **Cold start fully hidden:** `open_wait_ms` p95 **0**, 100 % warm hits.
- **The price:** pre-opened sessions idled p50 7.3 min vs a 21 s episode → **∼95 % idle**.
- **Reward sparse** for a 4B policy: 4 solves / 382; 42 % of episodes pass ≥ 1 test.

Next: `TB_PREOPEN_BATCHES=1` (cost), an easy-first curriculum or partial-credit reward (signal).

Source: `demo_cpu_terminalbench` AWS smoke, us-west-2, 2026-09-23; `tasks/arn_by_image.json`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 4: the warmstart probe measures pre-open and heartbeat in ∼7 min for a few cents

Use case covered.
Any rollout that opens a fresh sandbox per episode (a container launch each time), and any run whose sandboxes sit idle across the weight-sync pause.

Why this probe.
The Terminal-Bench smoke showed pre-open works but wastes ∼95 % of booted time — too costly to tune at full scale (∼70 min, 229 Runtimes). The probe answers *how much does pre-open save?* and *does a heartbeat keep a session alive?* in **∼7 min for a few cents**. A probe folder inside `demo_cpu_terminalbench`, not a demo: it tests that demo's two mechanisms.

Design considerations.

- **MockPolicy episodes**, no GPU / model / trainer → only the sandbox is measured.
- **A 60 s idle timeout** (training keeps the 15-min default) makes reclaim observable in minutes.
- **Both backends** (own Runtime and Code Interpreter): they behave differently under a heartbeat.
- **Pre-open depth** 1 batch here; `TB_PREOPEN_BATCHES` (default 2) in the real demo.

Source: `demo_cpu_terminalbench/warmstart/` probe design; Terminal-Bench smoke 2026-09-23; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 4: a look-ahead data source replays slime's deterministic prompt order to keep 64 episodes pre-booted

- **The problem:** slime hands the pool only `CONC` × engines episodes and draws the next batch only when this rollout ends — the pool alone cannot see ahead.
- **The fix:** slime's prompt order is **deterministic** (a shuffled dataset walked in order, fixed seed per epoch). A **look-ahead data source** (`tb_lookahead_source.py`, `--data-source-path`) replays it and, on every draw, tells the pool the episodes of the **next** `TB_PREOPEN_BATCHES` **batches**.
- **The pool** keeps `TB_PREOPEN_BATCHES` × RB × NS = 2 × 4 × 8 = **64** upcoming episodes booted, each on its own new session on its task's Runtime; a worker claims a ready one and the next starts booting.
- **Mispredictions** are trimmed at the next draw and open on demand. `TB_PREOPEN_BATCHES=0` turns it off.
- **Heartbeat:** a `SessionKeeper` pings (`exec("true")`) any session idle past `WARM_PING_FRACTION` × `WARM_IDLE_TIMEOUT` (0.5 × 900 s). Always on here; `HEARTBEAT=1` (default off) in every other demo.

Source: `demo_cpu_terminalbench` code (`tb_lookahead_source.py`, pool + `SessionKeeper` defaults); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 4: pre-open cut the wait p50 to 0–5 ms and the makespan by 15–20%, at the cost of idle held

2026-09-25, us-west-2: 24 MockPolicy episodes in batches of 6.

Table 15 **Pre-open vs cold open, both backends**

Backend	Mode	Wait p50	Waited	Idle held	Makespan
own Runtime	cold	774 ms	24/24	0 s	31.9 s
own Runtime	pre-open	**5 ms**	8/24	75 s	25.5 s (**−20 %**)
Code Interpreter	cold	1715 ms	24/24	0 s	28.7 s
Code Interpreter	pre-open	**0 ms**	6/24	70 s	24.3 s (**−15 %**)

- **Pre-open takes the boot off the critical path**; only the first batch pays it.
- **The cost is idle held** — Terminal-Bench's 95 % idle at scale → `TB_PREOPEN_BATCHES=1` next.

Source: `warmstart/` probe, us-west-2, 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 4: a heartbeat kept 3/3 own-Runtime sessions alive; without it they were silently reclaimed

3 sessions idle 150 s against a 60 s idle timeout.

Table 16 **Heartbeat on vs off, both backends**

Backend	Heartbeat	Survived	Resume
own Runtime	on (14 pings)	**3/3**	104 ms, files intact
own Runtime	off	0/3	**silently reclaimed**: same id, fresh empty sandbox
Code Interpreter	on or off	0/3	expires at its session timeout regardless

- **Only helps an own Runtime:** a reclaimed session *looks healthy* but lost its files — silent; so Terminal-Bench's keeper is always on.
- **No-op on the Code Interpreter:** `run.sh` turns it off; the lever is `AGENTCORE_CI_SESSION_TIMEOUT` (3600 s default, free: sessions stop with their episode).

Source: `warmstart/` heartbeat probe, us-west-2, 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 5: the harness tier must scale with inference and the sandbox, or it silently becomes the ceiling

Use case covered.

High-throughput rollout: a fleet that has scaled inference and the sandbox and must not hit the forgotten third tier, the harness. The decoupled harness tier (`harness_pool.py`) is the **default in every demo**; `SCALE=1 bash 5_run.sh` in `demo_cpu_tasks` runs it at **4 engines : 256 harness workers : 256 microVMs** (DeepScaleR, RB 32 × NS 8, TP1, `MODE=A`, CONC 64).

Why this demo.

(1) The harness must scale **along with** SGLang and the sandbox, or a few coroutines silently become the ceiling: `throughput = min(inference, sandbox, harness)`. (2) It should scale **with the sandbox, not as part of the trainer** — as in DeepSeek DSec, the loop runs in the episode's session. A preset, not a folder: same task and image, only the tier sizes change (`demo_cpu_scale_lanes` was retired into this preset, 2026-09-28).

Design considerations.

- **Joined per episode:** a free worker pulls it → takes a **fresh session** first → then the **least-loaded engine** (ties rotate, sticky for its turns). Nothing pinned (replaces hash pinning and per-engine workers).
- **Isolation:** fresh microVM per episode by default; `SANDBOX_SESSIONS=M` (opt-in) pools M sessions per image with a fresh `ep-<hex>` dir — not for OS-level tasks.
- **256 opens** at `SANDBOX_OPEN_RATE` 20/s (quota 25/s); client pools sized to 256; `RB` × `NS` = 256 episodes/step.

Source: `demo_cpu_tasks SCALE=1` preset (2026-09-28); AgentCore session-open quota 25/s; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 5: the 256-worker RL smoke ran 592 episodes to eval 0.27, and the live pool check spread load evenly

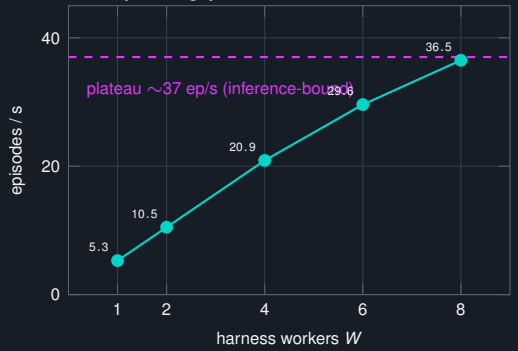
- **256-worker RL smoke** (the harness-scaling run, now `SCALE=1`; 2026-09-28): 592 episodes, 2 steps × 256; `raw_reward` 0.31 → 0.37, `eval/deepscaler` **0.27**.
- **Live pool check** (2026-09-25, no trainer, W = M = 64, real Runtimes): 1280 ops ok; engines **48** / **48** / **47** / **49**; a worker used up to 2 sessions and a session served up to 2 workers, so the tiers mix.
- **Old 4 × 4 × 4 lanes on AWS** (2026-09-23): 4 opens for 400 episodes; `raw_reward` 0.19–0.50, `eval/deepscaler` **0.358**.
- **Verifiable in logs (every demo):** every `[demo]` line carries `worker=`, `engine=`, `sandbox_session=`; the binding tier shows as `qdepth` (harness) / `sandbox_wait_ms` (sandbox) / `engine_inflight` (inference). `check_stickiness` reports sessions/engines per worker — a worker using several is expected.

Source: `demo_cpu_tasks SCALE=1` RL smoke 2026-09-28; live pool check 2026-09-25; old `demo_cpu_scale_lanes` AWS run 2026-09-23; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 5: the offline sweep shows the min() law — throughput grows with W, then plateaus at ∼37 ep/s

Figure 7 Offline sweep: throughput vs harness workers `W`



- **Offline sweep** (`demo_cpu_tasks/harness_scaling_demo.py`, seconds): throughput 5.3 / 10.5 / 20.9 / 29.6 / 36.5 ep/s at `W` = 1 / 2 / 4 / 6 / 8 while **harness-bound**.
- Then a **plateau ∼37 ep/s** once **inference-bound**.
- Unseeded sim — **shape, not absolute rate**.

Source: `demo_cpu_tasks/harness_scaling_demo.py` offline simulation (unseeded); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: a GPU comes only with an Instance; the farm's site and mode set how many episodes share it

- **GPUs exist on AgentCore only on capacity-provider Instances, and one session = one whole Instance.** The managed Code Interpreter has no GPU (`cuda_available=False`; `True` on a custom g6 Runtime).
- **Every site and mode trains KernelBench:** the policy writes a CUDA `ModelNew`; the checker asserts `allclose` vs the reference and reports a speedup. Farms serve from the box's own GPUs over the same `/invocations` contract.

Table 17 How many episodes share a GPU?

Demo	Episodes per GPU	Boundary between episodes
`demo_gpu_farm` · `LOCAL_GPU=0 FARM=whole`	one episode at a time on one warm remote `g6.xlarge` (1 slot)	no co-tenant, but the warm instance is reused (not a fresh VM per episode)
`demo_gpu_farm` · `FARM=fractional` (local or g6)	two per GPU (0.5 + 0.5), one instance	software, shared kernel + shared card (trusted-only)
`demo_gpu_farm` · `LOCAL_GPU=1 FARM=whole`	one whole GPU each (GPUs 6, 7), one shared instance	software, shared kernel, own card (trusted-only)

Source: GPU probe (`cuda_available`) on Code Interpreter vs custom g6 Runtime; AgentCore Instances docs [D5]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: demo_gpu_farm serves many sessions from ONE warm GPU instance, local or remote — trusted workloads only

Trusted-only: shared kernel + shared GPU — no MIG on L4 / L40S, a cooperative memory-fraction cap, no Landlock port, one shared uid on AgentCore. *(Merged 2026-09-26 from `demo_gpu_farm_fractional` + `demo_gpu_farm_whole`; 2026-09-28 `demo_gpu_single` merged in as `LOCAL_GPU=0`.)*

Table 18 **Farm slots by site and `FARM` mode**

site	`FARM=fractional` **(default)**	`FARM=whole`
`LOCAL_GPU=1` (default): this box's GPUs 6, 7	GPU 6 at 0.5 ⇒ **2 slots** on one card	GPUs 6, 7 at 1.0 ⇒ **2 slots**, one card each
`LOCAL_GPU=0`: remote `g6.xlarge` Instance (was `demo_gpu_single`)	L4 at 0.5 ⇒ **2 slots**	L4 at 1.0 ⇒ **1 slot**

- **Use case covered.** GPU-in-the-loop verification: RL on CUDA kernels (KernelBench [7]), where the reward requires running untrusted code on a real GPU, isolated from the training state.
- **Why this demo.** A GPU exists only on Instances, and a session is a whole instance, so two ids get two instances, not two GPUs, and a g6 per episode needs N instances at once (stalls on a starved fleet). **One shared `runtimeSessionId` + a per-episode `sub_session`** packs episodes onto one warm instance.

Source: `demo_gpu_farm` configuration (merged 2026-09-26 and 2026-09-28); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: the remote g6 pays its ∼207 s cold boot once; the local site leaves slime GPUs 0–5

Design considerations (`demo_gpu_farm`, slide 50).

- Ray leases **one warm-CUDA actor per** `sub_session`; memory cap cooperative (`B4_PER_SANDBOX_MEM_FRACTION=0.5`); `nvcc` `flock` + `fraction=0.5` mitigate the non-MIG compile wedge; `CONC` defaults to the slot count.
- `LOCAL_GPU=0`: capacity provider `acs_gpu_cp`, ARN required, `LOCAL_GPU=0` on every step 2–6; slime gets all 8 box GPUs (async save + `megatron_init`); the **∼207 s cold boot is paid once**, reopens ∼150–190 ms; torch pip-installed at container start (∼90 s) to stay under the image cap.
- `LOCAL_GPU=1`: slime on GPUs 0–5, the farm on 6, 7 (no g6 needed).
- `SNAPSHOT_EVERY=1`: application-level workspace snapshot to a store you own (`SNAPSHOT_DIR=demo_gpu_farm/snapshots`) — no usable VM/EBS GPU snapshot.

Source: `demo_gpu_farm` deploy config; `LOCAL_GPU=0` remote smoke 2026-09-25 (cold boot, reopen latency); checked 2026-09-28.

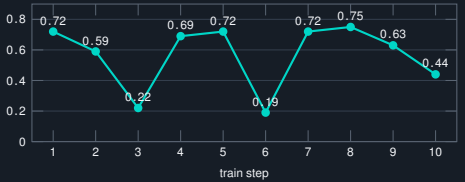
© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: both farm modes scored 0.65–0.775 on KernelBench; the spread is noise, not a mode difference

Table 19 `demo_gpu_farm` **runs, local site (remote g6: slide 53)**

Run	`eval/kernelbench`	Notes
`FARM=fractional` 2026-09-25	**0.65**	359 episodes split **181** / **178** across two actors on one GPU, lease wait 0; `exec` p50 9.0 s / p95 19.8 s, warm `open` 4.2 ms; PASSED 325, IMPORT_ERROR 18, NO_OUTPUT 16
`FARM=fractional` 2026-09-21	**0.775**	`Job succeeded`, ckpt iter 10
`FARM=whole` 2026-09-21 (×2)	**0.65** / **0.70**	`Job succeeded` each; logs in `demo_gpu_farm/logs/whole/`

Figure 8 `FARM=fractional` `raw_reward` **per step, 2026-09-25**



- `FARM=fractional` `raw_reward` 0.72, 0.59, 0.22, 0.69, 0.72, 0.19, 0.72, 0.75, 0.63, 0.44 (∼83 min, slime on GPUs 2–5).
- The spread is 20 problems × 2 samples — **noise, not a mode difference**. [PENDING: FARM=whole re-run in the merged folder].

Source: `demo_gpu_farm` runs 2026-09-21 and 2026-09-25 (logs in `demo_gpu_farm/logs/`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: the remote g6 site reached eval 0.65 and is sandbox-bound; its remote smoke failed on op=open 500s

`demo_gpu_farm` `LOCAL_GPU=0` (remote g6, formerly `demo_gpu_single`; logs in `demo_gpu_farm/logs/remote/`).

- **Committed 10-step run:** `raw_reward` ∼0.28–0.94, `eval/kernelbench` **0.65**; `exec` p50 8.5 s ≫ generation 2.5 s → **sandbox-bound**.
- **Earlier smokes:** GPU-farm 0.525; `SNAPSHOT_EVERY=1` 0.625 (3 files saved and restored per episode); custom-Runtime angle 0.65.
- **Remote smoke 2026-09-25** (`smoke-remote_20260925_113622`) completed **4 steps:**
  - `raw_reward` 0.531, 0.469, 0.25, 0.5625;
  - one 207 s cold boot, warm opens 150–190 ms.
- It then **failed** (rc=1) on `InvokeAgentRuntime` 500s during `op=open` retries: [PENDING: root cause of the remote-g6 op=open 500s].

Source: `demo_gpu_farm/logs/remote/`: committed 10-step run, earlier smokes, remote smoke 2026-09-25 (rc=1); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 6: the farm isolates in software on shared hardware — a hardware boundary needs one g6 per session

Host side = the CPU pool's uid mode (own uid, 0700 dir, `prlimit`, kill-and-scrub; the Landlock jail is not ported yet). The GPU adds the layers in the table.

`demo_gpu_farm` **is trusted-only.** `FARM=fractional` = software isolation on shared hardware; `FARM=whole` removes memory contention and the wedge; a **hardware boundary** needs one g6 per session, which no demo runs now.

Live check (2026-09-25, local farm on GPU 6): slot A and slot B ran as separate uids 20000 / 20001; B was denied A's files, dirs and home; `../`, absolute and symlink escapes were refused; 4 separate CUDA contexts, the 0.5 memory cap held in both slots; closing A left no process and removed its dir. On the AgentCore g6 (no capabilities) the farm falls back to one shared uid.

Table 20 GPU isolation on a shared instance

Layer	Enforced	Still shared
CUDA context	a separate context per actor process	the same physical GPU and driver
GPU memory	`B4_PER_SANDBOX_MEM_FRACTION` — **cooperative**	a process can bypass it
Caches	`SANDBOX_ISOLATE_HOME=1`: own CUDA / compile cache	default: shared warm cache (poisoning risk)
Faults	`nvcc` `flock`, `fraction=0.5`	**no MIG**: a wedge hits every co-tenant
Kernel / host	as the CPU pool	one kernel, one instance

Source: `demo_gpu_farm` local isolation check on GPU 6, 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Side note: a warm m5 pool cut cold boots from ∼32 per step to 2 per run, at eval 0.7875

Demoted to a side note (2026-09-26). The capacity-provider m5 Instances are AgentCore, but the **pooling** (many episodes sharing one warm Instance through `sub_sessions`) is **our own code**, and a shared Instance **can't give full isolation** — every episode shares one kernel. Keep it for trusted / RLVR-style workloads where an ∼83 s cold boot per episode is unaffordable; otherwise slim the > 2 GB image (or install deps at boot) so it fits a microVM.

- **How:** `SANDBOX=agentcore-ray-farm SANDBOX_POOL=N` mints N pool session ids per run → **N warm m5s** (`m5.xlarge`, ∼4 GB torch + sci image); cold boot paid **N times per run**, not once per episode; each episode is its own `sub_session` on the least-loaded member.
- **Why pool:** an Instance per episode paid ∼**32 cold boots per step** (375 total; `eval/deepscaler` 0.76).
- **Measured:** live probe **2 Instances, 2 cold boots** (83 s, in parallel), warm opens **121–328 ms**; RL smoke `smoke_pool2_20260925_113657`: 10 steps, 406 episodes on 2 Instances, **2 cold boots**, warm op p50 117 ms, step 38–219 s (vs ∼715–753 s), `eval/deepscaler` **0.7875**.

Source: `demo_cpu_instance_pool` live probe and RL smoke `smoke_pool2_20260925_113657` (2026-09-25); per-episode Instance baseline run; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Side note: a Landlock jail per exec passed 22/22 checks, but the kernel and more are still shared

Achieved (a Landlock jail per exec, +50 ms; **22/22** probe checks live on m5; 13 families still reward 1.0): writes **only its own dir** (siblings, farm root, `/tmp`, `/dev/shm`, `/root` → `EACCES`; dir deleted on close) · **no TCP** `connect`/`bind` (port 8080, IMDS, internet), no `AWS_*` · signals / `ptrace` / `/proc/<pid>` blocked across domains · subreaper kills the tree at exec end · watchdog 256 procs / 4 GiB RSS, 1 GiB files.

Table 21 **What the instance pool still shares**

Still shared	Consequence
Kernel	a kernel exploit or Landlock bypass defeats all of it — no VM boundary
uid / **PID ns**	all uid 0 in one userns; `ps` shows neighbours' command lines
UDP / **DNS**	not covered by Landlock
CPU / **RAM**	no per-episode cgroup; the RAM watchdog only polls
netns / **blast radius**	one network stack; a crash, OOM or full disk hits every episode

Source: `demo_cpu_instance_pool` Landlock probe on m5 (22/22 checks), 2026-09-28; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Side note: Landlock-grade isolation stops opportunistic reward hacking, not hostile code

Pitfalls

- Per-episode uids fail on AgentCore (**all capabilities dropped** → `chown` `EPERM`; local docker hides it — test with `--cap-drop ALL`).
- Landlock is kernel-ABI dependent (m5: ABI 7).
- `SANDBOX_FARM_SHM=allow` / `NET=allow` / `LANDLOCK=0` weaken it silently.
- No process survives across execs; no OS-mutating tasks.
- `demo_gpu_farm` is weaker still (no Landlock, one uid, no MIG).
- Warm Instances bill while idle.

In plain English: Landlock-grade isolation on a shared kernel — enough against accidental or opportunistic reward hacking, **not** a boundary against hostile code. **Full isolation = one AgentCore session per episode.**

Source: `demo_cpu_instance_pool` probes on m5 (Landlock ABI 7); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 7: demo_cpu_branching turns one episode into a tree of samples, so the update must stay unbiased

Use case covered.
A real coding-agent CLI (Claude Code) as the harness; one episode fans out into sub-agents and compacted contexts.

Why this demo.
Every other demo maps one episode to one sample; here an episode is a **tree of samples**, and the update must stay unbiased (slide 66).

Design considerations.

- **Rollout-level GRPO:** advantage and token-mean loss per episode.
- **CLI next to slime; only its tools in the sandbox** (node 22 + `claude`).
- `MODE=A`, `dp_size=1` (TP2 × PP2), sync checkpoint save.
- **The CLI needs the tool parser** (`qwen25` / `qwen3`) and **a per-turn cap above Qwen3's** `<think>` (`MAX_RESPONSE_LEN` 2048).

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 7: demo_cpu_branching runs end to end after three blocker fixes, verified without a trainer

Three blockers, all fixed:

1. **The CLI's root guard** — fixed with `IS_SANDBOX=1` (2026-09-19).
2. **One-turn episodes, reward 0** (2026-09-19 smoke: exit 0, `fan_out=1`): `run.sh` lacked the SGLang tool / reasoning parsers, so Qwen3's `<tool_call>` came back as a plain `end_turn` text reply and the CLI stopped. Fixed 2026-09-25.
3. **CLI exit 1, "response exceeded the output token maximum"** when a turn hit the 1024 per-turn cap (`stop_reason=max_tokens`). The cap is now `MAX_RESPONSE_LEN`, default 2048.

In plain English: verified without a trainer: the session starts, `claude` 2.1.278 runs as root, the dial-back to `:18021` works, the parser extracts the tool call, and the checker scores 0 / 0 / 1.

Source: `demo_cpu_branching` smokes 2026-09-19 and 2026-09-25; trainer-less verification; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Ch. 7: the branching RL smoke fans out and scores 0.47, but a fourth bug zeroed the gradient

RL smoke 2026-09-25 (NR=3: 2 train steps + 1 eval, job succeeded). `eval/swe` = **0.47**. Fan-out and reward work; **the gradient was zero**: `rollout_advantage.py` grouped by `sample.index`, which slime makes unique per rollout, so every group had one member. Fixed: group = `index // n_samples_per_prompt`, plus a regression test. Learning [PENDING: branching re-run with the advantage fix].

Table 22 `demo_cpu_branching` **RL smoke, per step**

Metric	Step 1	Step 2
Episodes (CLI `exit`≠0)	33 (0)	35 (0)
Leaf samples · `fan_out` mean / p50 / max	193 · 5.8 / 4 / 37	167 · 4.8 / 4 / 10
`rollout/raw_reward`	0.61	0.48
Sandbox op p50 / p95	88 / 136 ms	87 / 135 ms
Step time (actor train)	49 min (30 min)	32 min (24 min)
`advantages` · `pg_loss` · `grad_norm`	**0 · 0 · 0**	**0 · 0 · 0**

Source: `demo_cpu_branching` RL smoke 2026-09-25 (NR=3); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

The episode's shape, not its domain, decides what you scale

The task's shape — not the domain — sets the config. **One microVM serves many families**: the sandbox is task-agnostic (`write_file` / `exec` / `read_file`); task specificity is *files seeded per episode*, not a new image.

- `--rollout-max-response-len` — per-turn generation budget; set from the family's p95 per-turn response (math 1k–32k; code 100–800; CUDA kernel 200–1k; tool call 50–300).
- `EPISODE_MAX_TURNS` (default 3) — **the single biggest lever on episode cost**: single-turn → 1; iterative code/SQL → 3–5; long-horizon SWE/web → 15–50+.

Table 23 **Episode regime → bottleneck → what to scale**

Regime	Example	Bottleneck	Scale
single-turn, long trace	math, reasoning	**generation**	inference
few-turn, heavy checker	KernelBench (CUDA)	**sandbox**	sandbox / GPU
multi-turn, big observations	SWE, RAG, web	context growth	both + warm session

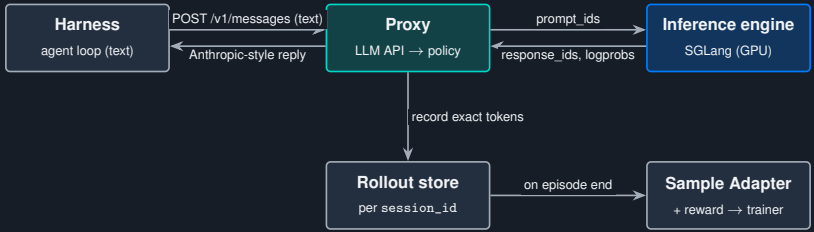
Source: Project per-family response-length and turn settings (demo configs); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

The proxy records the exact sampled tokens (TITO), so the trainer learns from what was really sampled

A harness works in **text**; RL trains on **token IDs**. Re-tokenizing decoded text can drift from what the model actually sampled — corrupting the action, the importance ratio, and the loss mask.

Figure 9 **Token-in-token-out through the proxy**



Token-in-token-out (TITO): the proxy records the **exact sampled** `token_ids` **+ logprobs** at generation time and forwards them intact.

- The proxy is the only component straddling both sides (text ↔ tokens), so it is the only correct place to capture them.
- For binary RLVR rewards this token-level exactness is not optional — **slop silently corrupts the signal**.

© 2026, Amazon Web Services, Inc. or its affiliates.

The demos fix trainer, inference and the harness tier, and change only the sandbox ARN and the task

On the 8×L40S box the demos **fix trainer, inference, and the harness tier** and change only the sandbox ARN + task — the reference-architecture disaggregation made concrete.

Table 24 **The scaling demo (1/3): the concrete setup**

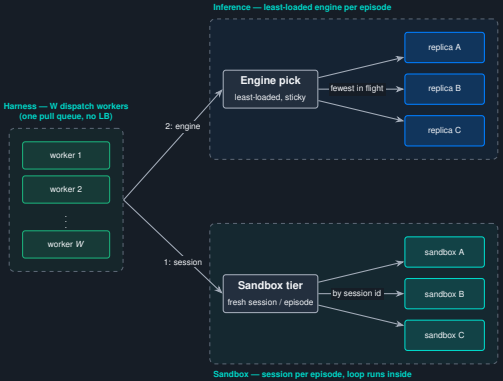
Component	Where	Config (shared)
Trainer	4 GPUs	Megatron, GRPO, `global_batch_size=32`, `ACTOR_GPUS=4`
Inference	4 GPUs	SGLang, 2 engines (TP2), `ROLLOUT_GPUS=4`; each episode on the **least-loaded engine**, sticky
Harness	**in the sandbox session**	`mini_claude` loop per session; trainer keeps `HARNESS_WORKERS` = RB × NS dispatch workers + one adapter per engine
Sandbox	**remote AgentCore**	CPU microVM (arm64) **or** GPU `g6` (amd64); one session/episode

Source: Shared demo configuration (`run.sh` / `demo.env`) on the 8×L40S trainer box; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Three tiers are sized independently and joined per episode — sandbox first, then engine

Figure 10 The scaling demo (2/3): the three tiers (`harness_pool.py`, default in every demo)



Three tiers, **sized independently** and joined per episode (sandbox first, then engine).

© 2026, Amazon Web Services, Inc. or its affiliates.

Scale two tiers and the bottleneck moves to the third — diagnose before you scale

The min() law — `throughput = min(inference, sandbox, harness)`**.** Scale two and the bottleneck moves to the third.

- **Diagnose first:** compare `perf/sglang_gen_ms_*` vs `perf/sandbox_ms_*`. Generation dominates → add SGLang replicas; sandbox dominates → add sessions / ARNs / the GPU farm.
- **The harness tier** (every demo; `demo_cpu_tasks SCALE=1` = 4 engines : 256 workers : 256 microVMs) makes all three dials **independent pools**, including the **easily-forgotten third**: the harness. It must scale **along with** inference and the sandbox, or a few coroutines become the ceiling; and it scales **with the sandbox, not the trainer** (the loop runs in the session, as in DSec). What stays trainer-side is IO dispatch + adapter tokenisation (the GIL is the part to watch at large W).
- **Load-balancing differs by side:** inference → the **least-loaded engine** per episode (sticky for its turns, so the prefix cache stays hot); sandbox → the **session id *is* the isolation unit** (routed by `task.image`). You never stand up your own load balancer.

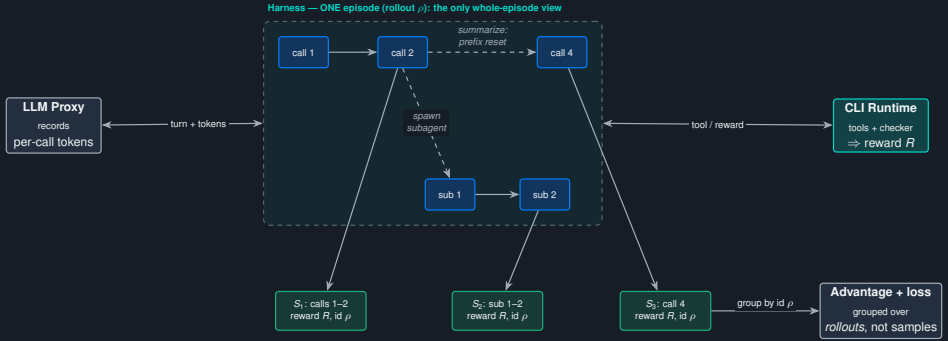
The scaling demo (3/3). Measured sweep: slide 48.

© 2026, Amazon Web Services, Inc. or its affiliates.

A real agent fans one episode into many samples, so advantage and loss must be normalized per rollout

The reconciliation `cpu_branching` implements, and the reason a **real** agent needs it. A production agent (Claude Code CLI) **fans one episode out into many samples** — it spawns subagents (the `Task` tool) and summarizes context (resetting the token prefix). Scored naively per-sample, that fan-out silently reweights the GRPO baseline and the loss [1].

Figure 11 **One episode → many samples, grouped by rollout id ρ**



Source: Agent Lightning [1] (slide 72); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Tagging every sample with a shared rollout id and terminal reward keeps an incidental fan-out from reweighting the update

Solution (the reconciliation `cpu_branching` implements; diagram slide 66):

- The CLI runs **inside a CLI-capable CPU Runtime** (image bakes node22 + `claude`).
- One episode returns a `list[Sample]`, all tagged with the shared `rollout_id` ρ and the episode's terminal reward R.
- **Advantage baseline + loss are normalized per rollout, not per sample** (token-mean loss), so an incidental fan-out can't reweight the update.
- `MODE=A`, `dp=1` (TP2×PP2), sync save.

In plain English: `demo_cpu_branching` (§5, Ch. 7, slide 60) implements this; its three blockers are fixed and verified without a trainer — RL smoke fans out (p50 4) and scores (eval 0.47); an all-zero-advantage bug is fixed, learning [PENDING: branching re-run with the advantage fix].

Source: Agent Lightning [1] (slide 72); `demo_cpu_branching` RL smoke 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Heartbeat prevents the loss of a warm session; snapshot recovers from it

Two levers a long RL run composes, demonstrated by `HEARTBEAT=1` and `demo_gpu_farm SNAPSHOT_EVERY=1` (§5). **Use case:** any multi-hour run whose warm, stateful sessions (installed deps, seeded repo) must survive both the periodic pause and a genuine instance recycle.

- **Heartbeat** (`HEARTBEAT=1`, any demo on an own Runtime; a no-op on the Code Interpreter — Ch. 4, slide 45) — during weight sync generation stalls; an idle session is **reclaimed (∼15 min)** [D3] and the next round pays a cold start. A `SessionKeeper` pings on a timer, **resetting the idle clock**, so the *same* warm microVM survives. Lives **outside the harness** (`KeptSandbox`), because the stall is a training-loop property, not an episode one.
- **Snapshot** (`demo_gpu_farm SNAPSHOT_EVERY=1`) — a recycled GPU instance comes back blank (no VM/EBS snapshot), so the workspace is serialized to a store you own (S3) and restored onto a fresh session.

The pairing: *heartbeat prevents loss; snapshot recovers from it.*

Source: AgentCore Runtime docs [D3] (idle ∼15 min); `warmstart/` heartbeat probe 2026-09-25; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

A day-1 pilot needs one generic Runtime and one hook; plan around session, image and GPU limits

Day-1 pilot (a few hours)

1. Deploy **one generic-image Runtime** (arm64, VPC, `CreateAgentRuntime` [D9]; `SANDBOX=agentcore-runtime`) — the agent loop runs in its sessions.
2. Point slime at it with the `--custom-generate-function-path` hook — **no fork of slime**. Only if the task needs its own environment: build a task image and route by `task.image`.
3. Seed a task family, run the standard 10-step loop, watch `eval/<task>` and the `[sbx-op]` / `[sglang-gen]` latencies.
4. Flip **CPU↔GPU** with one env var when your checker needs CUDA.

Limits & quotas to plan around

- **Session length:** 8 h on a CPU microVM, 14 d on a GPU Instance; **idle ∼15 min → reclaimed** (use `HEARTBEAT=1` to survive the weight-sync pause).
- **Image size:** ∼2 GB arm64 cap — install heavy deps at `CMD`-time, or (trusted workloads only) run it on a warm CPU Instance pool (`demo_cpu_instance_pool`, side note).
- **GPU sharing: no MIG** on L4 / L40S — cross-session GPU sharing is software-only (the Ray farm), not hardware-partitioned.
- **Next step after training:** serve the resulting policy on Bedrock / SageMaker — the trained checkpoint is a standard model artifact.

Source: AgentCore docs [D3], [D5], [D9] (slide 74); project image-size measurement; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

One image, N sessions, one hook: AgentCore is the isolated per-episode sandbox, slime does the learning

- **Can:** isolated per-session sandbox for untrusted output (CPU default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, **GPU on Instances**; a one-hook slime integration with the harness running in the sandbox session.
- **Cannot:** a microVM has no GPU; no cross-session GPU fractioning (no MIG on L4); a session can't exceed 8 h without moving to Instances (14 d).
- **How to run:** one generic-image Runtime (or a task image / GPU capacity provider when needed) → seed a task → the agent loop runs in the session → in-sandbox checker gives the reward → slime trains via one `generate` hook. **CPU↔GPU is one env var.**

In plain English: one image, N sessions, one hook: AgentCore is the isolated per-episode sandbox; slime does the learning.

Source: AgentCore docs [D3], [D5] (slide 74); project demos (Section 5); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

References: papers, models & tech reports · RL algorithms & systems · AgentCore docs, SDK & CLI

References — papers, models & tech reports

- **[1] Agent Lightning** — harnessed agentic RL · https://arxiv.org/abs/2508.03680
- **[2] DeepSeek-R1** · https://arxiv.org/abs/2501.12948
- **[3] Qwen3 Technical Report** · https://arxiv.org/abs/2505.09388
- **[4] Llama-Nemotron** · https://arxiv.org/abs/2505.00949
- **[5] Kimi K2** · https://arxiv.org/abs/2507.20534
- **[7] KernelBench** · https://huggingface.co/datasets/ScalingIntelligence/KernelBench
- **[8] MBPP** · https://arxiv.org/abs/2108.07732
- **[9] slime** — the trainer · https://github.com/THUDM/slime · docs https://thudm.github.io/slime/
- **[27] SGLang / RadixAttention** · https://arxiv.org/abs/2312.07104 · cache-aware router https://lmsys.org/blog/2024-12-04-sglang-v0-4/

© 2026, Amazon Web Services, Inc. or its affiliates.

References — RL algorithms & systems (further reading)

- **[13] GRPO (DeepSeekMath)** · https://arxiv.org/abs/2402.03300
- **[14] DAPO** · https://arxiv.org/abs/2503.14476
- **[15] GSPO** · https://arxiv.org/abs/2507.18071
- **[18] AReaL** — asynchronous RL system · https://arxiv.org/abs/2505.24298
- **[19] HybridFlow / veRL** · https://arxiv.org/abs/2409.19256
- **[22] Truncated Importance Sampling** — train/inference mismatch · https://fengyao.notion.site/off-policy-rl
- **[28] APRIL** — active partial rollouts · https://arxiv.org/abs/2509.18521

© 2026, Amazon Web Services, Inc. or its affiliates.

References — AgentCore docs, SDK & CLI

- **[D2] Host with Runtime** — isolation, compute types · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html
- **[D3] Runtime — how it works** — idle 15 m / max 8 h · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html
- **[D5] Instances (GPU)** · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html
- **[D8] InvokeAgentRuntime** · https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html
- **[D9] CreateAgentRuntime** · https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html
- **[D13] Observability** — vCPU / GB-hour metrics · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html
- **[D15] Python SDK** — `@app.entrypoint` · https://github.com/aws/bedrock-agentcore-sdk-python
- **[D16] AgentCore CLI** — `@aws/agentcore` · https://www.npmjs.com/package/@aws/agentcore
- **[D17] Runtime service contract** — :8080, /invocations, /ping · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS